Practical No.: 4

Name: More Vaishnavi Dadaso

Roll No.: 269

Problem Statement: Implement a binary search tree and perform the following operations:

a. Insert (Handle insertion of duplicate entry)

b. Delete

c. Search

d. Display tree (Traversal)

e. Display - Depth of tree

f. Display Mirror image g. Create a copy

h. Display all parent nodes with their child nodes

i. Display leaf nodes

j. Display tree level-wise

Code:

```
from collections import deque
import copy


class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BST:
    def __init__(self):
        self.root = None

    # Insert
    def insert(self, node, data):
```

```
    if node is None:
        return Node(data)

    if data < node.data:
        node.left = self.insert(node.left, data)

    elif data > node.data:
        node.right = self.insert(node.right, data)

    else:
        print("Duplicate Entry Not Allowed")

    return node

# Search
def search(self, node, key):
    if node is None:
        return False

    if node.data == key:
        return True

    if key < node.data:
        return self.search(node.left, key)

    return self.search(node.right, key)

# Find Minimum
def minimum(self, node):
    while node.left:
        node = node.left
```

```
    return node

# Delete
def delete(self, node, key):
    if node is None:
        return node

    if key < node.data:
        node.left = self.delete(node.left, key)

    elif key > node.data:
        node.right = self.delete(node.right, key)

    else:
        if node.left is None:
            return node.right

        elif node.right is None:
            return node.left

        temp = self.minimum(node.right)
        node.data = temp.data
        node.right = self.delete(node.right, temp.data)

    return node

# Inorder
def inorder(self, node):
    if node:
        self.inorder(node.left)
        print(node.data, end=" ")
```

```
        self.inorder(node.right)

    # Preorder
    def preorder(self, node):
        if node:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    # Postorder
    def postorder(self, node):
        if node:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")

    # Depth
    def depth(self, node):
        if node is None:
            return 0

        return max(
            self.depth(node.left),
            self.depth(node.right)
        ) + 1

    # Mirror
    def mirror(self, node):
        if node:
            node.left, node.right = node.right, node.left
            self.mirror(node.left)
```

```
        self.mirror(node.right)

    # Parent Child
    def parentChild(self, node):
        if node:
            print("Parent:", node.data, end=" -> ")

            if node.left:
                print("Left:", node.left.data, end=" ")

            if node.right:
                print("Right:", node.right.data, end=" ")

            print()

            self.parentChild(node.left)
            self.parentChild(node.right)

    # Leaf Nodes
    def leafNodes(self, node):
        if node:
            if node.left is None and node.right is None:
                print(node.data, end=" ")

            self.leafNodes(node.left)
            self.leafNodes(node.right)

    # Level Order
    def levelOrder(self):
        if self.root is None:
            return
```

```
        q = deque()
        q.append(self.root)

        while q:
            node = q.popleft()

            print(node.data, end=" ")

            if node.left:
                q.append(node.left)

            if node.right:
                q.append(node.right)

# Main Program
bst = BST()

while True:
    print("\n1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder")
    print("5. Preorder")
    print("6. Postorder")
    print("7. Depth")
    print("8. Mirror")
    print("9. Copy Tree")
    print("10. Parent Child")
    print("11. Leaf Nodes")
    print("12. Level Order")
```

```
print("13. Exit")

ch = int(input("Enter Choice : "))

if ch == 1:
   x = int(input("Enter Number : "))
   bst.root = bst.insert(bst.root, x)

elif ch == 2:
   x = int(input("Delete Node : "))
   bst.root = bst.delete(bst.root, x)

elif ch == 3:
   x = int(input("Search : "))

   if bst.search(bst.root, x):
      print("Found")
   else:
      print("Not Found")

elif ch == 4:
   bst.inorder(bst.root)
   print()

elif ch == 5:
   bst.preorder(bst.root)
   print()

elif ch == 6:
   bst.postorder(bst.root)
   print()
```

```
    elif ch == 7:
        print("Depth =", bst.depth(bst.root))

    elif ch == 8:
        bst.mirror(bst.root)
        print("Mirror Created")

    elif ch == 9:
        newTree = copy.deepcopy(bst)
        print("Tree Copied Successfully")

    elif ch == 10:
        bst.parentChild(bst.root)

    elif ch == 11:
        bst.leafNodes(bst.root)
        print()

    elif ch == 12:
        bst.levelOrder()
        print()

    elif ch == 13:
        break

    else:
        print("Invalid Choice")
```

Output:

1. Insert

2. Delete

3. Search

4. Inorder

5. Preorder

6. Postorder

7. Depth

8. Mirror

9. Copy Tree

10. Parent Child

11. Leaf Nodes

12. Level Order

13. Exit

Enter Choice : 1

Enter Number : 34

1. Insert

2. Delete

3. Search

4. Inorder

5. Preorder

6. Postorder

7. Depth

8. Mirror

9. Copy Tree

10. Parent Child

11. Leaf Nodes

12. Level Order

13. Exit

Enter Choice : 1

Enter Number : 56

1. Insert

2. Delete

3. Search

4. Inorder

5. Preorder

6. Postorder

7. Depth

8. Mirror

9. Copy Tree

10. Parent Child

11. Leaf Nodes

12. Level Order

13. Exit

Enter Choice : 45

Invalid Choice

1. Insert

2. Delete

3. Search

4. Inorder

5. Preorder

6. Postorder

7. Depth

8. Mirror

9. Copy Tree

10. Parent Child

11. Leaf Nodes

12. Level Order

13. Exit

Enter Choice : 4

34 56

1. Insert

2. Delete

3. Search

4. Inorder

5. Preorder

6. Postorder

7. Depth

8. Mirror

9. Copy Tree

10. Parent Child

11. Leaf Nodes

12. Level Order

13. Exit

Enter Choice : 5

34 56

1. Insert

2. Delete

3. Search

4. Inorder

5. Preorder

6. Postorder

7. Depth

8. Mirror

9. Copy Tree

10. Parent Child

11. Leaf Nodes

12. Level Order

13. Exit

Enter Choice : 6

56 34

1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Depth
8. Mirror
9. Copy Tree
10. Parent Child
11. Leaf Nodes
12. Level Order
13. Exit

Enter Choice : 7

Depth = 2

1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Depth
8. Mirror
9. Copy Tree
10. Parent Child
11. Leaf Nodes
12. Level Order
13. Exit

Enter Choice : 8

Mirror Created

1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Depth
8. Mirror
9. Copy Tree
10. Parent Child
11. Leaf Nodes
12. Level Order
13. Exit

Enter Choice : 9

Tree Copied Successfully

1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Depth
8. Mirror
9. Copy Tree
10. Parent Child
11. Leaf Nodes
12. Level Order

13. Exit

Enter Choice : 10

Parent: 34 -> Left: 56

Parent: 56 ->

1. Insert

2. Delete

3. Search

4. Inorder

5. Preorder

6. Postorder

7. Depth

8. Mirror

9. Copy Tree

10. Parent Child

11. Leaf Nodes

12. Level Order

13. Exit

Enter Choice : 11

56

1. Insert

2. Delete

3. Search

4. Inorder

5. Preorder

6. Postorder

7. Depth

8. Mirror

9. Copy Tree

10. Parent Child

11. Leaf Nodes

12. Level Order

13. Exit

Enter Choice : 12

34 56

1. Insert

2. Delete

3. Search

4. Inorder

5. Preorder

6. Postorder

7. Depth

8. Mirror

9. Copy Tree

10. Parent Child

11. Leaf Nodes

12. Level Order

13. Exit

Enter Choice : 2

Delete Node : 13

1. Insert

2. Delete

3. Search

4. Inorder

5. Preorder

6. Postorder

7. Depth

8. Mirror

9. Copy Tree

10. Parent Child

11. Leaf Nodes

12. Level Order

13. Exit

Enter C**hoice :**